Reflected XSS with Event Handlers and Href Attributes Blocked

1. Overview

Field	Details
Vulnerability	Reflected Cross-Site Scripting
Category	HTML Injection
Injection Context	HTML Markup
Security Restrictions	Event Handlers and href Attributes
Severity	Context-dependent
Lab Platform	PortSwigger Web Security Academy

2. Description

This vulnerability occurs when an application reflects user-controlled input into an HTML document while applying restrictions to potentially dangerous HTML attributes.

The application blocks event-handler attributes and the `href` attribute, attempting to prevent JavaScript execution.

However, restricting specific attributes does not necessarily make HTML injection safe. Other browser-supported HTML elements, attributes, and execution contexts may introduce additional attack surfaces.

This lab demonstrates the importance of understanding browser parsing behavior and applying context-aware output encoding instead of relying exclusively on attribute blacklists.

3. Root Cause

The root cause is incomplete HTML sanitization.

Potential contributing factors include:

- Reliance on attribute-based blacklists.
- Insufficient HTML output encoding.
- Unsafe reflection of user-controlled input.
- Failure to account for alternative HTML parsing and execution contexts.

4. Proof of Concept (PoC)

Authorized lab testing methodology:

1. Open the assigned lab.
2. Identify the parameter that reflects user-controlled input.
3. Inspect the generated HTML.
4. Determine which event-handler attributes are blocked.
5. Check whether `href` attributes are removed or modified.
6. Examine how the browser interprets the remaining HTML elements and attributes.
7. Identify whether an alternative permitted markup context can produce script execution.
8. Verify using a harmless proof-of-execution in the lab.

Example HTML element for context inspection:

```
None
<div>USER_INPUT</div>
```

This illustrates a generic HTML reflection context and is not a confirmed working payload.

5. Security Impact

Successful exploitation may result in:

- JavaScript execution in the victim's browser.
- Manipulation of webpage content.
- Unauthorized actions through an authenticated session.
- Exposure of information accessible to client-side scripts.
- Phishing and malicious redirection opportunities.

6. Recommended Remediation

- Use context-aware output encoding.
- Avoid inserting untrusted input directly into HTML.
- Use established HTML sanitization libraries when HTML input is required.
- Configure sanitizers using an explicit allowlist.
- Remove unsafe elements and attributes.

- Prefer safe DOM APIs such as `textContent`.
- Apply a restrictive Content Security Policy.
- Test sanitization against browser parsing edge cases.

7. Verification

1. Repeat the original test cases.
2. Verify that untrusted HTML is safely encoded or sanitized.
3. Confirm that prohibited attributes cannot be reintroduced.
4. Test alternate markup and malformed HTML.
5. Verify normal rendering functionality.

8. References

- PortSwigger: HTML Context XSS
- OWASP: Cross-Site Scripting Prevention Cheat Sheet
- MDN: HTML Sanitizer API

9. Disclaimer

For educational purposes and authorized security testing only.